

1493

Engineering Copilot: Real AI Workflows for Vault, BOMs, and Manufacturing Data

Christian Gessner
COOLORANGE S.r.l.

Tyler Warner
IMAGINiT Technologies

Learning Objectives

- Identify practical use cases where AI enhances engineering workflows in Vault environments
- Design an architecture combining Vault data, APIs, and AI services for real-world scenarios
- Implement AI-assisted workflows such as EBOM-to-MBOM transformation and validation
- Assess limitations, risks, and best practices when integrating AI into engineering systems

Description

AI is everywhere, but how does it actually help engineering teams working with CAD, BOMs, and Vault?

This class demonstrates real-world AI applications on top of Autodesk Vault, with live examples showing how EBOM and MBOM data is interpreted, transformed, and enriched by AI while engineers stay in control.

It covers natural-language interfaces, rule-based automation, and AI-assisted decisions for BOM transformation, process planning, and validation, plus the architectures, integration patterns, and limitations behind them.

Speaker(s)

Christian Gessner is a co-founder and Head of Research & Innovation at COOLORANGE, where he drives research into technologies that help customers automate and customize Autodesk CAD, PDM, and PLM solutions and integrate them with enterprise systems. He has over 25 years of full-stack development experience specializing in Autodesk product data and lifecycle management, and before founding COOLORANGE he was a member of the data management software engineering team at Autodesk.

- <https://www.coolorange.com>
- <https://www.linkedin.com/in/christian-gessner>

Tyler Warner is a software developer at IMAGINiT Technologies, where he builds integrations across the Autodesk portfolio on Autodesk Platform Services, connecting design and engineering data to the enterprise systems around it.

He works across web and desktop applications, and builds platform tooling that takes the repetitive work off an engineering team - automating design processes, streamlining workflows, and moving data between systems that were never designed to talk to each other.

He is a top contributor to Autodesk's open-source projects and shares what he learns with the developer community.

- <https://www.imaginit.com>
- <https://www.linkedin.com/in/tylerwarner33>

About this Handout

The class is a demonstration: in thirty minutes it establishes a shared vocabulary, shows nine working examples, and closes with an honest account of limitations and risks, staying at the level of *what this does*.

This handout is for the people who build it - **software developers, system integrators, and technically minded CAD/PDM administrators** - and where the presentation names a tool, it gives the endpoint, the auth flow, and the limit that will bite you.

It answers six questions the demos raise but cannot address on stage:

Question	Where
How do the pieces - app, agent, LLM, MCP, RAG - fit together in one request?	1.2, 5.2
How do I expose Vault as tools without handing an LLM my Vault credentials?	3.3
What does a well-designed tool surface for Vault and ERP data look like?	3.3, E.2, E.4
How do I build a retrieval pipeline over engineering standards that returns the right clause?	4.2
Where does the human sit in the agent loop, and what do I log so an approval survives an audit?	5.2, 7.5
What does this cost, and why does the cost grow the way it does?	7.4

TABLE 1. THE QUESTIONS THIS HANDOUT ANSWERS, AND WHERE.

The version anchors this handout was written against: MCP specification 2026-07-28, Vault Data API v2, Vault 2027.1, and Microsoft Agent Framework 1.0.

Section 3.2 summarizes what changed in that MCP revision; the wire format itself stays in the specification, linked from Appendix D.

What this is not. Several of the nine examples are prototypes or concepts, stated per example.

None of this recommends putting an unsupervised model in front of production engineering data - the reference architecture in Section 1.2 exists precisely because that would be a bad idea.

Table of Contents

Part 1. Foundations	5
1.1 Four Primitives, Precisely Defined.....	5
1.2 The Reference Architecture	6
1.3 Deterministic Logic vs. Probabilistic Reasoning	6
Part 2. LLM.....	7
2.1 What it is.....	7
2.2 How it works.....	7
2.3 In the Autodesk context	8
2.4 Go deeper	9
Part 3. MCP	9
3.1 What it is.....	9
3.2 How it works.....	10
What changed in the 2026-07-28 specification.....	10
3.3 In the Autodesk context	11
Authorization and Identity.....	11
The two-layer problem	11
APS access patterns available today.....	12
Patterns on the roadmap.....	12
What this means for a Vault MCP server.....	13
MCP Apps (SEP-1865): Interactive UI	14
Wrapping the Vault Data API as MCP Tools	14
3.4 What trips people up.....	16
A Tool Surface Is a Design Problem, Not a Wrapper	16
Threat Model and Controls.....	16
3.5 Go deeper	16
Part 4. RAG	17
4.1 What it is.....	17
4.2 How it works.....	17
The Indexing Pipeline.....	18
The Retrieval Pipeline	18
Grounding, Citation, and Evaluation.....	18
4.3 In the Autodesk context	19
4.4 What trips people up.....	20
4.5 Go deeper	20
Part 5. Agents.....	20
5.1 What it is.....	20
5.2 How it works.....	20
The Agent Loop	20
Multi-Agent Decomposition: Planner, Reviewer, Executor.....	21
Human-in-the-Loop: The Diff Pattern	21
Tool Design, Error Handling, and Loop Control	22
5.3 In the Autodesk context	22
5.4 Go deeper	23
Part 6. Patterns from the Field.....	23
6.1 Reading the Capability Labels.....	23

6.2 Conversational Assistant Grounded in Your Engineering Data	24
6.3 Exposing a System of Record as MCP Tools - Read Surface vs Write Surface	24
6.4 Starting on an Administrative Surface	25
6.5 Propose and Approve Through a Reviewable Diff	25
6.6 Graph and Relationship Traversal for Change Impact Analysis	25
6.7 Splitting Deterministic Rule Checks from Interpretive Judgment	26
6.8 Write-Back to ERP or PDM with Guardrails	26
6.9 Federating a Third-Party MCP Server into a Host Assistant	26
6.10 Demo-to-Concept Map	27
Part 7. Developer Toolkit	27
7.1 Autodesk Building Blocks	27
Autodesk Assistant	27
APS authentication: two identity models	27
7.2 Frameworks and SDKs	29
First-party APS building blocks	29
Other components you will need	29
Giving an agent an API it cannot call: dll2llm	30
7.3 Model Selection: Hosted, Local, and Model-Agnostic Design	30
7.4 Cost Mechanics	31
Six controls that actually work	31
7.5 Governance and the Audit Trail	32
Part 8. Getting Started	32
8.1 Choosing the First Use Case	32
8.2 A Sensible Sequence	33
Appendix A - MCP Server Review Checklist	34
A.1 Tool surface	34
A.2 Safety	34
A.3 Operations	34
A.4 If the server ships an MCP App	35
Appendix B - Reference Architectures	36
B.1 MBOM Editor, end to end	36
B.2 AI Drawing Checker	36
B.3 ForgeFlow change impact analysis	36
Appendix C - Glossary	37
Appendix D - Links and References	38
Model Context Protocol	38
Autodesk - MCP and AI	38
Autodesk - platform and data	39
Frameworks and RAG	40
Security	40
Products shown	40
Appendix E - Developer Reference	41
E.1 Field Notes from Shipping an APS Viewer MCP App	41
E.2 Designing an MCP Tool Surface: A Case Study	42
E.3 Risks and Controls	43

MCP attack vectors.....	43
Production risks.....	44
E.4 Vault Data API Tool Design.....	45
Viewables - two failure modes.....	45
Turning that into good tools	46

Part 1. Foundations

1.1 Four Primitives, Precisely Defined

Four words carry the whole class, and each is routinely used imprecisely.

Terms this handout uses but does not define where they first appear are in the glossary, Appendix C.

Term	One-line definition	Responsible for
LLM	A neural network that predicts tokens; it understands language and reasons over what you put in front of it	Thinking. Nothing else.
Agent	Software with a goal and a set of tools, where the LLM decides the next call and the agent executes it	Coordination and control flow
MCP	An open protocol for exposing tools and data to an LLM-based client	Connecting to live systems
RAG	Retrieval-Augmented Generation: search your documents, put the hits in the prompt	Connecting to knowledge

TABLE 2. THE FOUR PRIMITIVES, AND WHAT EACH IS RESPONSIBLE FOR.

MCP commonly provides structured access to live systems and can expose actions; RAG commonly retrieves relevant passages from an indexed corpus. Neither guarantees correctness or freshness: an MCP tool can return cached, stale, incomplete, or incorrect data, while a well-maintained retrieval system can return exact and current facts. In this handout, use MCP primarily for transactional system-of-record queries and actions, and RAG primarily for document knowledge. Most real workflows need both.

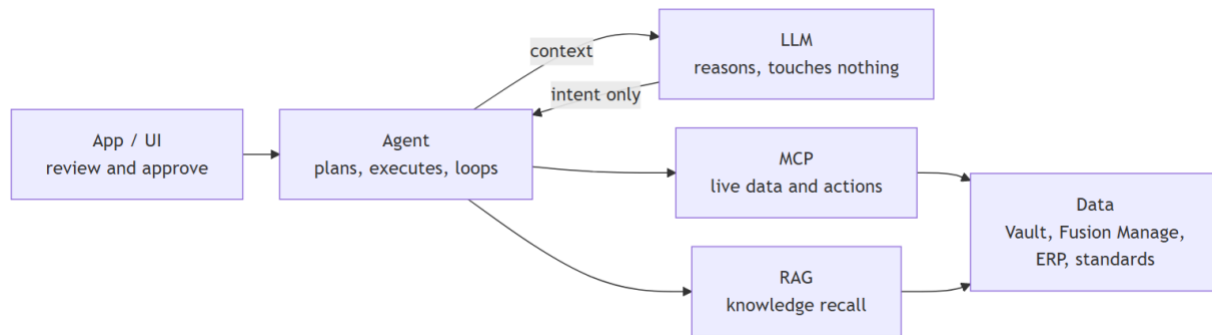
"What is the current lifecycle state of item 9000-0000?" is an MCP question - one right answer, and it changes.

"What does our drafting standard say about datum references?" is a RAG question - a paragraph in a PDF that changes once a year.

Answering the first from a periodically refreshed document index risks stale lifecycle data and lacks transactional semantics. Implementing every standards question as a custom MCP tool, on the other hand, creates an unnecessarily large maintenance surface.

1.2 The Reference Architecture

Every example in this class instantiates the same five-layer stack.



The LLM never touches Vault.

It receives context and emits an intent; the agent is the only component that calls MCP and RAG, and the only one holding a credential.

That separation buys three things:

- **Groundedness and citability.** Every factual claim traces to a tool result or a retrieved chunk.
- **Enforceable permissions.** The MCP server authenticates as a real identity with real Vault permissions. The model inherits nothing it was not granted.
- **Substitutability.** Swap the LLM without touching Vault; add a Vault capability without touching the AI side.

One request runs left to right and loops back to the agent step until the goal is met; the loop itself is diagrammed in Section 5.2. A read-only, low-risk answer can return to the UI after normal authorization and validation checks. A state-changing or otherwise high-risk result is presented as a proposal and passes through human review before a deterministic system applies it.

The loop is what makes it an agent rather than a pipeline. The human review gate is what makes it deployable.

1.3 Deterministic Logic vs. Probabilistic Reasoning

	Deterministic (rule-based)	AI reasoning (pattern-based)
Definition	Rules explicitly defined	Patterns learned from examples
Repeatability	Same input, same output	Same input may produce different output
Predictability	100%	Probabilistic

	Deterministic (rule-based)	AI reasoning (pattern-based)
Validation	Easy - assert the rule	Requires evaluation; human review for high-risk decisions
Best at	Compliance, business rules, execution	Ambiguity, interpretation, drafting
Characteristic statement	"If A, then B"	"Based on context, B is likely"
Example in this class	The Vault PDF publish job	The EBOM-to-MBOM operation suggestion

TABLE 3. DETERMINISTIC LOGIC VERSUS AI REASONING.

The rule that governs every architecture in this document:

AI helps the engineer decide. A human approves. Deterministic systems execute the approved decision.

AI proposes the MBOM. The engineer approves it. Vault and the ERP carry out the approved result. Never the other way around.

Two corollaries developers get wrong:

1. **Do not use an LLM where a rule will do.** "If the material is 1.4404, the routing includes stainless deburring" is a rule - faster, free, testable, explainable. Use the model where genuine ambiguity lives: mapping a free-text customer note to a routing family, interpreting a drawing note, ranking candidate work centres.
2. **Wrap the model's output in deterministic guardrails.** Do not ask a human to catch what a foreign-key check catches for free.

Part 2. LLM

2.1 What it is

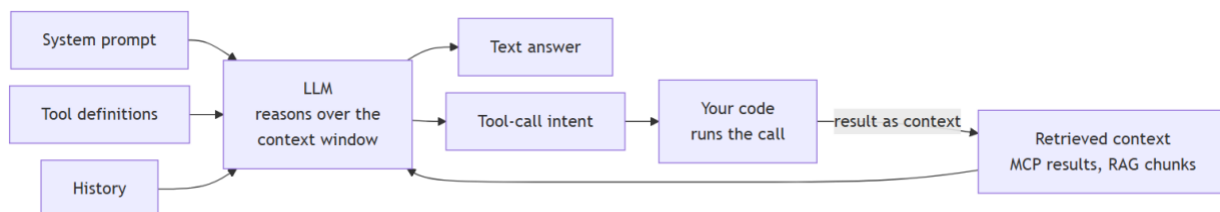
A large language model is a neural network that predicts the next token in a sequence, trained on a large text corpus, able to follow instructions and reason over the text you put in front of it.

It has never seen your Vault, your Fusion Manage tenant, or your ERP: everything it knows about your business arrives through MCP or RAG on each call.

It is not a database, not deterministic, not always right, and not autonomous on its own.

2.2 How it works

Four properties matter for behaviour, cost, and failure modes.



Tokens and the context window.

Everything the model sees on a call - system prompt, tool definitions, history, retrieved documents, tool results - must fit a fixed context window, near 1M tokens on current frontier models and far smaller locally.

Measure a serialized BOM with the provider's token-counting endpoint rather than estimating from file size.

The context window is not durable private memory. The base model does not preserve facts between independent calls; conversation state must be supplied again or managed by the application or model provider. Re-sending a continuously growing history is one way agent cost compounds (Section 7.4).

Non-determinism.

The same input can produce different output, and even at temperature zero, batching and floating-point non-associativity make bit-identical reproduction unreliable.

LLM output cannot be your system of record, and it cannot be your audit trail.

What is auditable is the deterministic record of what was proposed, what a human approved, and what a deterministic system wrote.

Tool calling.

Given tool definitions with JSON Schema parameters, the model emits a structured call block rather than prose, and executes nothing: your code executes the intent and returns the result. This is the mechanism underneath MCP and agents.

Structured output and constrained decoding.

All major providers now enforce a supplied JSON Schema at the sampling level, so output parses by construction; Anthropic exposes this as a response-format control and `strict: true` on a tool definition.

Syntactic validity is therefore solved and semantic validity is not: a schema-valid MBOM operation can still name a work centre that does not exist, so enforcement moves the failure from "unparseable" to "plausible and wrong", which is harder to detect.

Generator-critic checks - a second model pass judging the first against ground truth the generator does not hold (Section 5.2) - reduce the rate but do not change the category, so validate every value against master data before a human sees it.

Mark everything `required`, set `additionalProperties: false`, and use `enum` for every closed value set - a permissive schema validates garbage.

2.3 In the Autodesk context

Vault's own data shapes are hostile to a context window in specific, measurable ways.

A released assembly exported through the Data API carries a `masterId`, an `id`, an `iterationId`, and a full property-definition GUID beside every value, so the GUIDs and envelope routinely outweigh the engineering content several times over - project the response and resolve GUIDs to display names server-side before the tokens are spent (Appendix E.4).

A multi-level BOM is worse: a full explosion of a mid-sized machine returns tens of thousands of rows, which is why `get_bom` defaults to one or two levels rather than complete depth.

The model also has no notion of which Vault property is authoritative, so an item carrying both a CAD-driven `Material` and a hand-typed ERP `Material` will be reasoned over as if the two were equal - decide the precedence in the tool layer, not in the prompt.

Vault lifecycle state is the sharpest case of the model's staleness: a released state read three turns ago may be superseded by the time the agent acts on it, so any state-dependent action re-reads before it proposes.

And because a Vault item description, a drawing note, and an ERP error message all arrive as ordinary text in the context window, everything Vault stores is potential instruction as well as data (Section 3.4).

2.4 Go deeper

- Token counting - <https://platform.claude.com/docs/en/build-with-claude/token-counting>
- Structured outputs - <https://platform.claude.com/docs/en/build-with-claude/structured-outputs> - and Strict tool use - <https://platform.claude.com/docs/en/agents-and-tools/tool-use/strict-tool-use>
- Models overview - <https://platform.claude.com/docs/en/about-claude/models/overview> - per-model context-window sizes
- Pricing - <https://platform.claude.com/docs/en/about-claude/pricing> - the figures Section 7.4 relies on
- OpenAI structured outputs - <https://platform.openai.com/docs/guides/structured-outputs>

Part 3. MCP

3.1 What it is

An open protocol, first published in late 2024 and now supported by every major AI provider, that standardizes how an AI application calls external tools and reads external data.

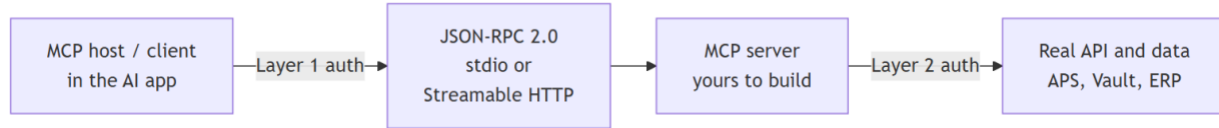
The "USB-C for AI integrations" analogy holds in the way that matters: you build one server, and every compliant client can use it.

Role	Where it lives	Responsibility
MCP Host / Client	Inside the AI application (Claude Desktop, Copilot, your own chat app, an IDE)	Discovers tools, translates the model's intent into calls, returns results to the model
MCP Protocol	The wire	JSON-RPC 2.0 messages: requests, responses, notifications
MCP Server	Yours to build	Exposes data and logic as callable tools; validates inputs; calls the real API; returns results

TABLE 4. THE THREE MCP ROLES, AND WHERE EACH LIVES.

3.2 How it works

MCP runs on JSON-RPC 2.0 over two current transports: **stdio** for a local subprocess, and **Streamable HTTP** for shared and cloud servers. The older HTTP+SSE transport is deprecated.



A server declares three kinds of capability.

Primitive	Controlled by	Use for
Tools	The model (with host/user consent)	Actions and parameterized queries - <code>get_item</code> , <code>create_operation</code> , <code>query_jobs</code>
Resources	The application	Context the app attaches - a file, a record, a document
Prompts	The user	Reusable templated workflows the user invokes deliberately

TABLE 5. THE THREE CAPABILITY KINDS AN MCP SERVER DECLARES, AND WHO CONTROLS EACH.

In the authors' experience, engineering integrations are almost entirely tools.

Each tool declares a `name`, a `description`, and an `inputSchema` (JSON Schema), and the description is prompt text the model reads to decide whether to call the tool, not documentation.

Full wire-format detail is in the specification (Appendix D). Build against the current spec, use an official SDK, and do not hand-roll the transport.

What changed in the 2026-07-28 specification

Stateless protocol core.

The `initialize/initialized` handshake and the `Mcp-Session-Id` header are gone, and each request travels independently, carrying its protocol version and client capabilities in `_meta`, so MCP now deploys behind an ordinary round-robin load balancer with no shared session store.

A document that still mentions `Mcp-Session-Id` predates this revision.

Multi Round-Trip Requests (MRTR).

Server-initiated requests no longer need a held-open bidirectional stream.

The server returns an `InputRequiredResult` (`resultType: "input_required"`) whose `inputRequests` field lists what it still needs; the client retries the original call with the answers in `inputResponses`, and a server correlating a long interaction across retries carries its own token in `requestState`.

This is the mechanism for a mid-tool-call approval prompt.

Three more changes reach into server code: `server/discover` is now mandatory, `ping` and `logging/setLevel` are removed with Roots, Sampling, and Logging deprecated, and `subscriptions/listen` replaces the HTTP GET endpoint and `resources/subscribe`.

The rest - header-based routing, cacheable list results (`ttlMs` and `cacheScope`, which keep the prompt-cache prefix stable, Section 7.4), authorization hardening, and an extensions framework - are in the release notes (Appendix D).

3.3 In the Autodesk context

Authorization and Identity

OAuth 2.1 baseline.

Authorization is optional in the specification, but a remote MCP server that authorizes at all must do so as an OAuth 2.1 resource server: PKCE for all clients, no implicit grant, tightened redirect handling, and Resource Indicators (RFC 8707) binding each token to the MCP server it was issued for.

The 2026-07-28 revision adds RFC 9207 issuer validation and deprecates Dynamic Client Registration (RFC 7591) in favour of Client ID Metadata Documents (CIMD).

Token passthrough is forbidden.

An MCP server must not forward the caller's token to a downstream API; use token exchange (RFC 8693) to mint a credential scoped to the target audience.

Otherwise a token issued for MCP server A can be replayed against system B - a confused-deputy escalation.

The two-layer problem

An MCP server fronting a cloud API has **two** independent authentication layers.

Layer	Between	The MCP server's role
Layer 1	MCP client and MCP server	OAuth resource server. It validates the token the client presents, or accepts no token at all on a local stdio transport
Layer 2	MCP server and the downstream API (APS, Vault, ERP)	OAuth client. It obtains its <i>own</i> credential for that API

TABLE 6. THE TWO AUTHENTICATION LAYERS, AND THE SERVER'S ROLE IN EACH.

Every design decision is one mechanism per layer, and the Layer 1 token never becomes the Layer 2 token.

Autodesk publishes reference implementations of the combinations below in **aps-mcp-auth-examples** (Appendix D).

APS access patterns available today

	A - 2-legged OAuth	B - Secure Service Account	C - Server-initiated login + 3-legged
Layer 1	Local stdio: none; remote HTTP: authenticated principal through a trusted IdP	Local stdio: none; remote HTTP: authenticated principal through a trusted IdP	Local stdio: none; remote HTTP: authenticated principal through a trusted IdP
Layer 2	Client credentials, app identity	SSA JWT assertion, service-account identity	Standard APS 3- legged OAuth, real user identity
Reaches the user's own data?	No	No	Yes
Best for	OSS, Model Derivative, Design Automation	APIs needing user context where no human is present	Interactive assistants over user-scoped data

TABLE 7. APS AUTHENTICATION PATTERNS AVAILABLE TO THIRD-PARTY MCP SERVERS TODAY.

Pattern A - 2-legged. An app-only token. Not recommended for new integrations.

Pattern B - SSA. The server exchanges a signed JWT assertion for an APS token in a service-account context - the right answer for unattended work.

Its limitation: the service account is a separate identity that sees only what it owns or what is shared with it, and one account shared into every user's projects leaks every user's data to every other user.

Pattern C - server-initiated login. The server hands the user an APS authorization URL, the user logs in in their own browser, and the OAuth callback lands back on the MCP server, which caches the tokens per session, so the APS credential never transits the MCP client and the passthrough prohibition is satisfied by construction.

Deliver the login URL as a plain link in a tool result, or through URL mode elicitation - an MRTR input request whose answer is the user visiting a URL rather than typing a value.

The cost is a stateful server that persists tokens per user and handles refresh rotation.

Pattern C - session security. A session identifier that unlocks cached APS tokens is a bearer credential unless the server binds it to an authenticated principal. Generate opaque session handles from a CSPRNG with at least 128 bits of entropy, bind each handle server-side to the verified Layer 1 identity and the originating authorization flow, give it a short expiry, rotate it after login and token refresh, and reject reuse by another identity. A client fingerprint may be used as a secondary anomaly signal, but not as authentication. Keep APS refresh tokens in an encrypted server-side credential store; do not place them in client-held session state. Where stateless operation is required, prefer token exchange into a short-lived, audience-restricted credential.

Patterns on the roadmap

Autodesk is building a central **MCP Auth Server** implementing the MCP authorization spec including CIMD.

It is not yet publicly documented, and the following reflects the authors' preview access rather than published APS material, so the endpoints may change.

On top of it, **Pattern D** calls APS 2-legged with an `x-user-id` impersonation header, and **Pattern E** performs an RFC 8693 token exchange into an audience-restricted APS token for the same user.

Pattern E is the architecturally correct one - the token is cryptographically derived, not asserted through a spoofable header - and it makes the server stateless again.

The MCP Auth Server is currently limited to Autodesk's own servers; a Pattern C server moves to Pattern E by replacing its Layer 2 adapter.

What this means for a Vault MCP server

When the model calls `get_item`, whose permissions apply? Two defensible answers, one indefensible one.

Pattern	Mechanism	When to use
User identity (3-legged)	The engineer signs in with their Autodesk ID via APS OAuth; the MCP server acts as them	Interactive assistants. Vault ACLs apply exactly as they do in the Vault client - the user cannot see through the AI what they cannot see in Vault. The default you should reach for.
Service identity (SSA)	The server authenticates as an APS Secure Service Account , mapped to a Vault user with least-privilege permissions	Unattended work: the drawing checker, scheduled analysis, a server-side agent with no human present
Shared admin credentials in a config file	-	Never. This is the confused-deputy vulnerability, pre-installed.

TABLE 8. IDENTITY PATTERNS FOR A VAULT MCP SERVER, AND THE ONE TO AVOID.

Never build an integration on a named person's account.

They change role or leave, the account is disabled, and the workflow stops; until then every audit-log entry is attributed to a human who did not perform the action.

Least privilege is easy here. The Vault user behind the SSA gets exactly the folder and category permissions the workflow needs, so a drawing checker that only reads drawings and writes markups is incapable of purging a file.

OWASP lists *excessive agency* among the top risks for LLM applications, and the Agentic Top 10 builds on it with tool-misuse and identity-abuse entries; the mitigation is server-side authorization, least-privilege credentials, a constrained tool surface, and human approval for high-risk actions. A permissions dialog can support informed consent, but neither it nor a prompt instruction substitutes for enforcement.

SSA mechanics - account creation, the RSA key pair, the RS256 assertion, and rotation - are in Section 7.1.

MCP Apps (SEP-1865): Interactive UI

This extension lets an MCP server ship its own user interface into the AI client, so a chat answer can carry an interactive 3D preview of a Vault assembly instead of a paragraph.

SEP-1865 reached **Final**, though the extension spec itself is still published as a draft (Appendix D).

- UI content is an MCP resource with a `ui://` URI and MIME type `text/html;profile=mcp-app`, referenced from a tool through `_meta.ui.resourceUri` (the flat `_meta["ui/resourceUri"]` form is deprecated) with `visibility` controlling who can call it.
- The host renders the view in a sandboxed iframe under a Content Security Policy. The resource declares `_meta.ui.csp` with allowed `connectDomains`, `resourceDomains`, `frameDomains`, and `baseUriDomains`; omit it and the host applies a restrictive `default-src 'none'`.
- The iframe talks to the host over a `postMessage` bridge speaking JSON-RPC 2.0. No SDK is required.

visibility	Effect
<code>["model", "app"]</code> (default)	Both the agent and the app can call the tool
<code>["model"]</code>	Only the agent may call it
<code>["app"]</code>	Only the app may call it; it is hidden from the agent's tool list

TABLE 9. TOOL VISIBILITY SETTINGS AND THEIR EFFECT.

The `["app"]` case gives your UI a privileged operation the model must never invoke on its own - "commit these approved redlines to Vault" - which should only fire from a button a human pressed.

An APS Viewer embed inside an MCP App needs `connectDomains` and `resourceDomains` covering the Autodesk endpoints it loads geometry from. Get the CSP declaration wrong and you get a blank iframe with no obvious cause; six non-obvious problems are in Appendix E.1.

Wrapping the Vault Data API as MCP Tools

The AI never talks to Vault. It calls tools like `get_bom_items()` or `create_operation()`, and the server handles auth, permissions, validation, and the API call.

Vault Data API v2 is the REST interface shipping with the Vault server.

```
Direct:  http(s)://{server}/AutodeskDM/Services/api/vault/v2/
Gateway: https://{gateway-
id}.vg.autodesk.com/AutodeskDM/Services/api/vault/v2/
```

The **OpenAPI specification** is published at

`/AutodeskDM/Services/api/vault/v2/openapi-spec.yml`. Generate a typed client from it first; it is the fastest on-ramp.

Authentication method	Header form	Direct	Via Gateway
Vault Account credentials (session token)	Bearer V:{token-uuid}	✓	✓
Autodesk ID (APS 3-legged OAuth, Auth Code + PKCE)	Bearer {jwt}	✓	✓
Secure Service Account (SSA), mapped to a Vault user	Bearer {jwt}	Tested	Tested
Windows authentication (NTLM)	NTLM challenge	✓	X

TABLE 10. VAULT DATA API AUTHENTICATION METHODS. VAULT ACCOUNT, AUTODESK ID, AND WINDOWS/NTLM BEHAVIOR ARE DOCUMENTED; THE SSA ROW RECORDS THE AUTHORS' TESTED CONFIGURATION. ONLY WINDOWS/NTLM IS DOCUMENTED AS BLOCKED THROUGH VAULT GATEWAY.

Field note: the SSA result is behavior tested by the authors with Vault 2027.1; Autodesk's public Vault Data API documentation does not explicitly identify SSA as a supported Vault authentication method. Re-test this configuration after Vault, Gateway, or authentication changes.

Vault Gateway publishes the Data API beyond the network over an **outbound** HTTPS connection from the Vault server to the Autodesk cloud - no VPN, no inbound firewall rule. Three constraints to design around:

- **The Gateway is documented only as a REST entry point for the Data API.** Autodesk does not publish it as a transport for the Vault SDK, and the authors have not made the SDK work across it, so anything SDK-only must run inside the network: the cloud tier queues it, the on-prem tier does it.
- **The Gateway does not send permissive CORS headers, so a browser app must go through your own server-side proxy.** On a directly reachable on-premise server the authors have enabled CORS in IIS via a `<restapi>` element under `<connectivity.web><server>` in `web.config`; the proxy is the safer default.
- **The Gateway rate-limits, and the limit is shared.** All APIs it proxies share one limit of 40,000 requests per 5 minutes, and a violation returns HTTP 429. A single request or response payload above roughly 37.5 MB fails with HTTP 500 (Vault Data API, "Rate Limits and Quotas").

Pagination is cursor/continuation based (`nextUrl`, `cursorState`) on every collection resource - suited to infinite scrolling, not to jumping to page 7. Bound your own `limit` rather than relying on the server to cap it.

Viewables. DWF/DWFX files have SVF viewables generated by the Vault server, retrievable at `vaults/{vaultId}/file-versions/{id}/svf/bubble.json` and feedable to the APS Viewer - the mechanism behind previewing 3D models in the chat.

The documented sample for passing PDFs directly via `/file-versions/{id}/content` does not work as written; a working approach is in Appendix D.

Two viewer failure modes bite here - the sandboxed iframe has no Vault credential, and Vault answers `202 Accepted` while it translates SVF on demand - and both, plus the rules for turning Vault endpoints into question-shaped tools, are in Appendix E.4.

3.4 What trips people up

A Tool Surface Is a Design Problem, Not a Wrapper

A REST API wrapped one-endpoint-per-tool is not an MCP server.

What separates a mediocre server from an excellent one is tool granularity, description quality, and server-side aggregation: shape tools like the questions engineers ask, aggregate and rank server-side, and write each tool description as a navigation graph.

The Job Processor Cockpit MCP server is the worked example (Appendix E.2), and Appendix A turns its principles into a review checklist to run over a server before it reaches production data.

Threat Model and Controls

Table 27 summarizes six common MCP risks in this engineering context; it is not a complete MCP threat model. Prompt injection is one major cross-cutting risk because untrusted text can enter the model's context from Vault properties, drawing notes, ERP messages, email, retrieved documents, and tool results. A production review must also cover SSRF, state-handle theft and replay, OAuth mix-up and redirect attacks, malicious local servers or proxies, authorization-server URL validation, and scope minimization.

The durable controls are architectural: least-privilege credentials, a minimized tool and read-data surface, authenticated and request-bound state, controlled network egress, output and data-loss controls, human approval for state changes, and a correlated audit log. Human approval of writes does not prevent a successful injection from disclosing data the agent can read.

Six representative vectors and their controls are shown in Appendix E.3. Supplement them with the MCP security best-practices review linked in Section 3.5.

3.5 Go deeper

- MCP specification - <https://modelcontextprotocol.io/specification/> - redirects to the current revision; the 2026-07-28 release notes are at <https://blog.modelcontextprotocol.io/posts/2026-07-28/>
- MCP authorization - <https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization> - read it beside Security best practices - https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices#token-passthrough
- MRTR pattern - <https://modelcontextprotocol.io/specification/2026-07-28/basic/patterns/mrtr>
- MCP Apps extension spec - <https://github.com/modelcontextprotocol/ext-apps/blob/main/specification/draft/apps.mdx> - and SEP-1865 at <https://modelcontextprotocol.io/seps/1865-mcp-apps-interactive-user-interfaces-for-mcp>

- OWASP MCP Security Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/MCP_Security_Cheat_Sheet.html - Appendix E.3's vectors in security-review vocabulary
- RFCs 8707, 9207, 8693, and 9728 are in Appendix D. Read RFC 8693 first; it is the one you will implement.
- Autodesk `aps-mcp-auth-examples` - <https://github.com/autodesk-platform-services/aps-mcp-auth-examples> - a server per Layer 1 / Layer 2 combination in Table 7; the DevCon workshop at <https://autodesk-platform-services.github.io/mcp-devcon2026/> walks it in Node.js, .NET, or Python
- MCP Inspector - <https://github.com/modelcontextprotocol/inspector> - exercise a tool surface by hand before any model sees it

Part 4. RAG

RAG solves one problem: *the model has never read your documents*.

"We index the documents and look them up" hides about six engineering decisions that decide whether the system retrieves the right clause or a plausible neighbouring one.

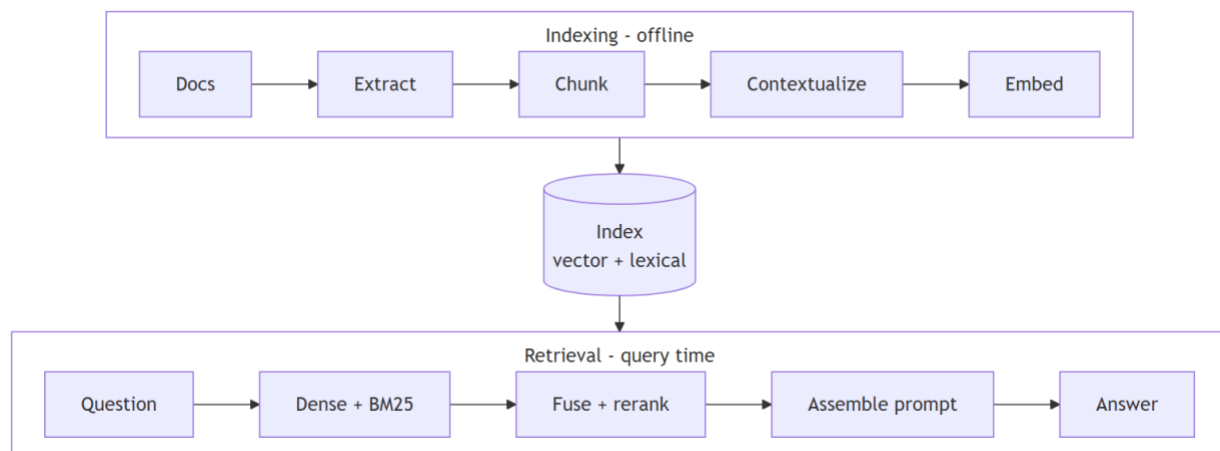
4.1 What it is

Retrieval-Augmented Generation: before the model answers, search a document corpus for passages relevant to the question and place them in the prompt, so the model reasons over supplied text instead of its training memory.

RAG is typically used to retrieve relevant prose rather than query transactional state. Its accuracy and currency depend on extraction quality, index refresh, retrieval configuration, and authorization design.

4.2 How it works

Two pipelines share one index: indexing runs offline to build it, retrieval runs at query time to answer from it.



The Indexing Pipeline

Extraction. A naive extractor turns a tolerance table into token soup, and no retrieval pipeline recovers what the extractor destroyed. Extract tables as structured markup.

Chunking. Split at clause and section boundaries, not every N characters, and carry document title, section, revision, and discipline into each chunk for filtering and citation.

Contextualize - the step worth adding. A chunk reading "The tolerance shall not exceed 0.05 mm" is nearly unretrievable because it names neither the feature nor the standard. Anthropic's **Contextual Retrieval** prepends a model-written explanation of each chunk before indexing:

Configuration	Top-20 retrieval failure rate	Reduction
Baseline (embeddings only)	5.7%	-
+ Contextual embeddings	3.7%	35%
+ Contextual BM25	2.9%	49%
+ Reranking	1.9%	67%

TABLE 11. ANTHROPIC'S PUBLISHED CONTEXTUAL RETRIEVAL RESULTS, BY TOP-20 FAILURE RATE.

A one-time cost per document revision for a two-thirds reduction in "the system did not find the rule".

Embedding. Changing embedding models means re-indexing the whole corpus, so version the index alongside the model name.

General-purpose models cover GD&T notation and material designations thinly, which is the argument for hybrid search: BM25 handles the exact token 1.4404 where a dense embedding places it near other stainless grades.

The Retrieval Pipeline

Hybrid, not either/or. Dense embeddings are good at semantics and bad at exact matches; BM25 is the inverse, and in engineering documents half of what you search for is an exact token. **Reciprocal Rank Fusion (RRF)** merges the two ranked lists on ranks, not scores, sidestepping the score-incompatibility that breaks weighted averaging.

Reranking. Pass each (query, document) pair from the fused top-N through a cross-encoder, which sees both at once. It is generally too expensive to run over the whole corpus, but suitable for reordering an application-specific candidate set, such as 50 candidates down to 5. Evaluate that configuration against your own golden set. Table 11 reports a separate Anthropic experiment that reranked 150 candidates and selected the top 20; in that experiment, reranking increased the reduction in top-20 retrieval failure rate from 49% to 67% relative to the baseline. That result must not be attributed directly to a 50-to-5 configuration.

Metadata filtering. Filter before you rank - current revisions, relevant discipline, applicable product line. A retrieved superseded standard is worse than nothing, because the model cites it confidently.

Grounding, Citation, and Evaluation

Cite, and make the citation checkable. Every drawing-checker finding carries the rule it rests on ("Std. 4.2"), verifiable in five seconds.

Instruct for abstention. Make "the provided documents do not answer this" an acceptable response, or the model synthesizes from the nearest chunk. This one instruction removes a large fraction of RAG hallucinations.

Evaluate with a golden set, built before the pipeline. It is the only thing that tells you whether a chunking change helped.

Layer	Metric	How
Retrieval	recall@k, MRR	A golden set of ~100 real questions with the ground-truth chunk(s) marked
Generation	Faithfulness / groundedness	Is every claim supported by a retrieved chunk?
Citation	Citation accuracy	Does the cited chunk actually contain the claim? Penalize hallucinated citations explicitly - a wrong citation is worse than none, because it defeats the human check
End-to-end	Task success	Did the reviewer accept the finding? Track accept/dismiss rates per rule; a rule with a 90% dismiss rate is a bug

TABLE 12. MINIMUM VIABLE RAG EVALUATION: ONE METRIC PER LAYER.

4.3 In the Autodesk context

In an Autodesk shop the corpus already lives in Vault, and that changes the pipeline in four concrete ways.

First, Vault versions everything, so index the released iteration of a standard rather than the tip - and make the Vault lifecycle transition to Released the event that triggers re-indexing, which turns "is the index current?" into a question the PDM system answers instead of a cron job.

Second, the chunk metadata you need for filtering is already curated in Vault: category, discipline, product line, and the revision number are Vault properties, so carry the item's `masterId` and its property values into each chunk and you get revision and discipline filters for free rather than parsing them out of the document text.

Third - and this is the one that fails a security review - Vault permissions do not automatically survive when content is copied into a search or vector index. Some retrieval platforms support document-level authorization or security trimming, but those controls must be explicitly configured and kept consistent with Vault. Store the Vault source entity ID and required authorization metadata on every chunk. At query time, restrict the candidate set to entities the calling identity may read and, where current access is important, revalidate that permission against Vault before ranking or returning content.

Fourth, Autodesk-generated PDFs are not ordinary documents: an Inventor or AutoCAD plot carries its title block, revision table, and notes as positioned text fragments rather than paragraphs, so a reading-order extractor interleaves the drawing notes with the title block and produces chunks that cite the wrong revision.

Extract the title block as structured fields first, attach it as chunk metadata, and index the notes separately.

4.4 What trips people up

RAG has a boundary. Four things it cannot do:

- **Current, transactional state.** "What is the lifecycle state of this file?" is an MCP question. Indexing Vault metadata into a vector store gives you a stale answer with no locking semantics.
- **Aggregation and counting.** "How many items use this material?" is a query, not a retrieval. Vectors do not count.
- **Precise numeric or structural reasoning.** BOM comparison, where-used traversal, quantity roll-up. Compute these in code and hand the model the result.
- **Anything requiring completeness.** Retrieval returns the top-K most similar chunks; it does not guarantee it returned *all* applicable rules. For a compliance check that gap is the whole problem, which is why the drawing checker is a **first reviewer** producing reviewable redlines, not a release gate.

4.5 Go deeper

- Contextual Retrieval - <https://www.anthropic.com/engineering/contextual-retrieval> - the source of Table 11, with the per-chunk context prompt and its cost
- Reciprocal Rank Fusion - <https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf> - Cormack, Clarke, Buttcher, SIGIR 2009
- Cross-encoder reranking - https://www.sbert.net/docs/cross_encoder/usage/usage.html - hosted alternative <https://docs.cohere.com/docs/rerank-overview>
- Hybrid search (dense + BM25) - <https://weaviate.io/developers/weaviate/search/hybrid>
- RAG evaluation - https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/ - context recall, faithfulness, and the golden-set model

Part 5. Agents

5.1 What it is

An agent is software with a goal and a set of tools, where the LLM decides the next tool call, the agent executes it, and the loop repeats until the goal is met or a human stops it.

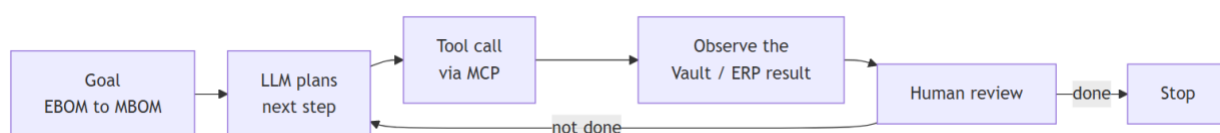
One thing separates an agent from a script: a script follows a fixed sequence, an agent decides the sequence.

If the sequence is known, write the script. Reach for an agent when the input is unstructured, the path genuinely varies, or the branches are unmanageable in code.

5.2 How it works

The Agent Loop

Key idea: the LLM decides which tools to call and in what order. The engineer stays in control at review checkpoints.



This is where the canonical rule of Section 1.3 becomes a control-flow decision. For state-changing or otherwise high-risk outputs in a regulated manufacturing environment, the human-review step is not optional. Read-only assistance can use the risk-based validation and oversight described in Section 1.2.

Multi-Agent Decomposition: Planner, Reviewer, Executor

The MBOM Editor (Section 6.5) uses **three cooperating agents** rather than one.

Agent	Job	Grounded by	Output
Planner	Drafts the sequence of steps to turn this EBOM into an MBOM	The BOM structure itself	An ordered plan
Reviewer / Grounder	Checks the plan against company knowledge and the real work centres from the ERP	RAG (manufacturing knowledge) + MCP (live ERP master data)	An annotated, corrected plan
Executor	Performs one approved step at a time	The approved plan only	Concrete changes, as a diff

TABLE 13. THE MBOM EDITOR'S THREE COOPERATING AGENTS.

The end-to-end flow is diagrammed in Appendix B.1.

The split buys three things.

Each agent has a narrow job and a focused prompt, so debugging "the planner proposed a non-existent operation" is tractable where "the agent did something wrong" is not.

The reviewer is where hallucinations die: it checks every proposed work centre against live ERP master data through MCP, so a hallucinated one fails a foreign-key check and never reaches the engineer.

And step-wise execution with a checkpoint bounds the blast radius to one rejected diff.

This mirrors the **generator-critic** pattern, and works here because the critic has ground truth the generator does not hold in context.

Do not reach for multi-agent by default. Every extra agent is another full context, another round trip, and a cost multiplier (Section 7.4). Split only when the sub-tasks have genuinely different tool surfaces and failure modes.

Human-in-the-Loop: The Diff Pattern

Every proposed change appears as a colour-coded diff the engineer accepts or rejects.

Five properties to copy exactly:

- **The AI never writes directly.** It produces a *proposed change set*; a deterministic system applies it after approval.
- **The unit of review is the individual change**, not the batch. Accept fourteen operations, reject one, edit another.
- **Pre-state and post-state are both visible.** Red/green, so review is fast enough that people actually do it.

- **Rejection is cheap and makes no change to Vault, ERP, or another system of record.** The rejected proposal and the reviewer's decision remain in the governance audit log. If rejecting is operationally expensive, reviewers stop rejecting - the rubber-stamping risk in Appendix E.3.
- **The approval is the auditable event.** Who approved, what exactly, when, and what the AI had proposed (Section 7.5).

When AI output feeds a system of record, the diff is the interface.

Tool Design, Error Handling, and Loop Control

Tool granularity. Match tools to *decisions*, not database operations.

`validate_mbom_against_erp()` is one call; `list_work_centers()` plus fifty `get_work_center()` calls is fifty round trips and fifty chances to derail.

Errors are results, not exceptions. "Work center 'SHEET-01' not found. Valid codes for this plant: SM-100, SM-200, SM-300." lets the agent self-correct in one turn; a 500 with a stack trace produces a hallucinated retry or a dead end.

Idempotency. Agents retry, so write tools must be safe to call twice - key on a client-supplied token, or make the operation an upsert.

Loop control - three limits, all mandatory:

Limit	Why
Maximum steps	An agent that cannot reach its goal will loop. Cap it and fail loudly rather than silently burning budget
Token / cost budget per run	Directly bounds the cost-unpredictability risk. Several platforms support this natively as a task budget the model is aware of, so it paces itself and finishes gracefully instead of being cut off mid-thought
Wall-clock timeout	For anything user-facing

TABLE 14. THE THREE MANDATORY LOOP-CONTROL LIMITS.

Confidence thresholds before any auto-action. Self-reported confidence is weak evidence; gate on structural signals - validation passed, two independent checks agreed, the retrieved chunk actually contained the rule - and route everything below the threshold to a human.

Log every step. Prompt, tool calls with arguments, tool results, model output, validation outcome, human decision. Your only debugging tool and your only audit trail (Section 7.5).

5.3 In the Autodesk context

Vault and Fusion Manage already contain most of the agent machinery, so the mistake is building a parallel one.

The human gate is the clearest case: a Vault lifecycle transition with a required approver, or a Fusion Manage workflow action with a transition permission, is an existing, permission-enforced, audited approval - route the agent's proposal into that transition rather than inventing an approval screen the quality auditor has never seen.

The deterministic executor is usually already there too, as the Vault Job Processor or an equivalent queue, so the agent's approved plan becomes a queued job with typed parameters and the loop ends at the enqueue, not at the write.

Two Vault behaviours will bite an agent that treats it as an ordinary REST API.

Vault check-out is an exclusive lock held by an identity, so an agent that checks a file out and then waits on an LLM turn blocks the engineer who owns the work - never hold a checkout across a model call, and design write tools to check out, write, and check in inside one deterministic step.

And every Vault history entry is attributed to the authenticating identity, so an SSA-backed agent appears in Vault history as one service user regardless of who asked: record your own correlation ID and the approving human's identity in the AI log, and write the correlation ID into the Vault comment on the transition so the two records can be joined at audit time (Section 7.5).

5.4 Go deeper

- Building effective agents - <https://www.anthropic.com/engineering/building-effective-agents> - the workflow-versus-agent distinction, tool design, and the "Evaluator-optimizer" section behind the generator-critic split. Companion: <https://www.anthropic.com/engineering/writing-tools-for-agents>
- Microsoft Agent Framework - <https://learn.microsoft.com/en-us/agent-framework/overview/> - orchestration primitives, session state, and MCP tool integration
- OWASP Top 10 for Agentic Applications 2026 - <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/> - the ASI01-ASI10 entries (Appendix E.3)

Part 6. Patterns from the Field

The nine class demos are instances of a smaller set of reusable patterns.

Each pattern below names the demo that showed it: use the demo as the concrete reference, and the pattern in your own build.

6.1 Reading the Capability Labels

The presentation tags every example with **capabilities** - shorthand for *what the AI is doing for the user*. They cut across the primitives: the same capability can come from MCP, RAG, or an agent loop.

Capability	The user's verb	What it means
Answer	Ask	Respond to a question in plain language, grounded in your data
Execute	Delegate	Carry out a bounded action after approval
Visualize	Explore	Render something inspectable - geometry, a chart, a diff
Validate	Check	Test a proposal against rules or master data before a human sees it

Capability	The user's verb	What it means
Learn	Teach	Absorb corrections and company knowledge so the next answer is better
Predict	Anticipate	Project forward from data - risk, impact, a maintenance window
Generate	Create	Draft new content: an MBOM, a routing, a quote, a redline
Motivate	Coach	Nudge behaviour at the moment it matters

TABLE 15. THE EIGHT CAPABILITY LABELS AND THE VERB BEHIND EACH. SECTIONS 6.2 TO 6.9 CARRY THE LABELS THEIR PATTERN DELIVERS.

Four properties are the acceptance criteria: **connected** (live systems of record), **context-aware** (grounded in the item, project, or page at hand), **human-guided** (a person approves what reaches a system of record), and **trusted** (permissions enforced, sources cited, actions logged). Run them over every pattern that follows: one you cannot answer all four for is a demo, not a deployment.

6.2 Conversational Assistant Grounded in Your Engineering Data

Put a natural-language interface in front of Vault or Fusion Manage so an occasional user describes what they want instead of learning menu paths and filter syntax.

Ground every answer in live permissions and page context, and keep a confirmation step before any action reaches the system of record.

Capabilities: Answer, Execute, Motivate.

Demonstrated by Moxy (a browser-extension chat avatar concept for Fusion Manage with gamified adoption credits, not a shipping product) and Autodesk Assistant Tech Preview in the Vault 2027.1 Thin Client (Section 7.1).

6.3 Exposing a System of Record as MCP Tools - Read Surface vs Write Surface

Wrap the API as question-shaped tools, not one tool per endpoint, and aggregate server-side in deterministic code so the model receives a small, already-correct result.

Keep reads broad; keep writes few, narrowly scoped, and returning a proposal for confirmation rather than performing the write.

Where an integration layer already exists, reuse it, so the customer's business rules and field mappings stay solved.

Capabilities: Answer, Validate, Execute.

Demonstrated by the Vault M

CP Server, the Job Processor Cockpit MCP, and the powerGate ERP Connector.

Piece	Choice
Server	ASP.NET Core over Streamable HTTP
Vault client	Typed client generated from the Vault Data API's OpenAPI specification
Viewer UI	Vite-bundled single file, served as the <code>ui://</code> resource
Derivative access	A <code>/svf</code> proxy carrying signed tickets, so the sandboxed iframe fetches without a Vault credential
Layer 1 auth	Autodesk OAuth sign-in, email claim checked against an allowlist that fails closed when empty
Layer 2 auth	Secure Service Account against Vault and APS

TABLE 16. THE VAULT MCP SERVER'S IMPLEMENTATION STACK, LAYER BY LAYER.

6.4 Starting on an Administrative Surface

Administrative surfaces are the best possible first use case: the data model is small, the operations are bounded, the tasks are tedious multi-click sequences, and mistakes are cheap and reversible, so nobody's released engineering data is at risk while you learn what your tool surface, guardrails, and review step need to be.

The same shape then carries to document knowledge - index standards ahead of time, retrieve the clause at query time, cite it checkably (Section 4.2) - which the AI Drawing Checker (Section 6.7) shows working against real standards.

Capabilities: Execute, Answer.

Demonstrated by IMAGINiT ACCelerate, an administration automation tool for Autodesk Construction Cloud - now Autodesk Forma - covering batch user management, RFI/issue/submittal reassignment, and automated audits.

6.5 Propose and Approve Through a Reviewable Diff

Every proposed change appears as a colour-coded diff the engineer accepts or rejects one change at a time; the AI never writes directly (Section 5.2 gives the five properties to copy exactly).

The same pattern reappears in this Part as the drawing checker's redlines, the ACCelerate confirm-before-act step, and the Autodesk Assistant's confirmation prompts.

Capabilities: Generate, Validate, Visualize.

Demonstrated by the Manufacturing BOM Editor (a web app between Fusion Manage and Odoo, with the planner/reviewer/executor split of Section 5.2).

6.6 Graph and Relationship Traversal for Change Impact Analysis

Let deterministic code walk the where-used graph - parent items, BOMs, assemblies, revision and lifecycle state - and let the model summarize and frame the risk.

The traversal is a query, not an inference; the AI contribution is turning a 200-row where-used result into "three released products are affected, one is in production, look here first". Cheap

deterministic checks alongside it catch what no human spots at row 140, such as a proposed substitute that is itself obsolete.

Capabilities: Predict, Answer, Visualize.

Demonstrated by Symetri ForgeFlow (reads a change order, follows the where-used trail through Fusion Manage, produces an impact summary; the reviewer still makes the call). Pipeline stages in Appendix B.3.

6.7 Splitting Deterministic Rule Checks from Interpretive Judgment

Run the checks that have exactly one right answer in code, run the interpretive checks through the model, then merge the findings into one result set.

A title-block revision mismatch is a deterministic comparison and should be 100% reliable; a missing GD&T datum reference is interpretive and belongs to the model. Mixing both into one prompt is the mistake.

Capabilities: Validate, Visualize, Learn.

Demonstrated by the AI Drawing Checker (a first reviewer producing reviewable redlines, each citing its rule; explicitly not a release gate, given RAG's completeness limitation in Section 4.4). Architecture and pipeline stages in Appendix B.2.

6.8 Write-Back to ERP or PDM with Guardrails

Expose curated tools with validated inputs and typed results, never raw system access or a `run_sql` tool, and validate every proposed value against real master data before a human is shown it.

Make writes idempotent, and keep the guardrails in the integration layer so they apply identically whichever client calls - Vault, Inventor, or an agent - and every ERP quirk you already solved stays solved.

Capabilities: Execute, Validate.

Demonstrated by the powerGate ERP Connector for AI (reuses the existing coolOrange powerGate Server and its PowerShell business rules) and the Manufacturing BOM Editor.

6.9 Federating a Third-Party MCP Server into a Host Assistant

A well-designed MCP server is client-agnostic by construction, so the same Vault or ERP server that serves your own app today serves a host assistant when that door opens, with no rework.

Autodesk has stated it is "building a path for certified third-party MCPs published on the Design and Make Marketplace to be callable directly inside Autodesk Assistant" (Appendix D), which sets two gates - certification, and Marketplace publication.

Capabilities: Answer, Execute.

Demonstrated by the announced third-party MCP support inside the Autodesk Assistant - one assistant orchestrating PLM, ERP, and maintenance systems, asked from inside Vault.

6.10 Demo-to-Concept Map

A reader who saw the class can find the demo they remember here.

Demo	Concepts it shows
Moxy	6.2 conversational assistant
Autodesk Assistant in Vault	6.2 conversational assistant; 6.9 federating third-party MCP
Vault MCP Server	6.3 system of record as MCP tools
IMAGINiT ACCelerate	6.4 starting on an administrative surface
Manufacturing BOM Editor	6.5 reviewable diff; 6.8 write-back with guardrails
Symetri ForgeFlow	6.6 graph traversal for change impact
Job Processor Cockpit MCP	6.3 system of record as MCP tools
AI Drawing Checker	6.4 retrieval over standards and documents; 6.7 deterministic vs interpretive checks
powerGate ERP Connector	6.3 system of record as MCP tools; 6.8 write-back with guardrails

TABLE 17. THE NINE DEMOS, MAPPED TO THE CONCEPTS THEY DEMONSTRATE. SECTION 6.9'S FEDERATION PATTERN HAS NO DEMO OF ITS OWN.

Part 7. Developer Toolkit

7.1 Autodesk Building Blocks

Autodesk Assistant

Built-in AI chat, shipping in Vault as **Autodesk Assistant Tech Preview in the Vault 2027.1 Thin Client** (Vault Professional), with **no custom code required**.

The 2027 release material describes contextual product help plus natural-language and RegEx search; the 2027.1 material adds checking metadata against standards, creating charts and visual reports, copying an item, building a folder structure, and lifecycle actions (Appendix D).

Autodesk's positioning is that interactions respect Vault permissions and data stays in the customer environment.

APS authentication: two identity models

The choice is architectural (Section 3.3).

Autodesk ID - three-legged OAuth (Authorization Code + PKCE) is for interactive applications where the human is present; Vault ACLs apply as the signed-in user, who must be assigned an Autodesk ID in the Vault User Profile settings.

Secure Service Accounts (SSA) are machine identities for unattended work:

Step	Call
Create service account	POST /authentication/v2/service-accounts
Generate RSA key pair	POST /authentication/v2/service-accounts/{serviceAccountId}/keys - the private key is returned exactly once
List keys	GET /authentication/v2/service-accounts/{serviceAccountId}/keys - returns all keys; there is no get-single-key operation
Enable, disable, or delete a key	PATCH / DELETE /authentication/v2/service-accounts/{serviceAccountId}/keys/{keyId}
Exchange JWT assertion for token	POST /authentication/v2/token with grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer, assertion={jwt}, and Basic authorization using client_id:client_secret

TABLE 18. THE SECURE SERVICE ACCOUNT LIFECYCLE. PATHS ARE RELATIVE TO [HTTPS://DEVELOPER.API.AUTODESK.COM](https://developer.api.autodesk.com/ssa/v1); SSA/V1 IS THE DOCUMENTATION PATH, NOT THE API BASE.

The assertion is signed **RS256**, kid matching the returned keyId, with required claims iss (Client ID), sub (service account UUID), aud (always <https://developer.api.autodesk.com/authentication/v2/token>), exp (0 to 5 minutes in the future - longer-lived assertions are rejected), and scope (a JSON array of strings such as ["data:read", "data:write"], not a space-separated string).

The documentation's example assertion also carries iat, though it is not listed among the required claims; jti is not required either.

Access tokens live **60 minutes**.

Do not hand-roll the assertion. Recent versions of **aps-sdk-net** ship a first-party Autodesk.SecureServiceAccount package covering the whole lifecycle, with equivalent support in **aps-sdk-node** (@aps_sdk/secure-service-account).

Key rotation is built in. Keys can be **disabled as well as deleted**: create the new key, cut over, disable the old one, confirm nothing broke, then delete it. Disabling is reversible; deleting is not. A service account holds at most three keys, so delete the retired key before the next rotation.

Store the private key in a secret manager - not source control, not a config file next to the binary, not the MCP server's JSON config in a home directory.

The **SSA Management Tool** (<https://ssa-manager.autodesk.io/>) creates accounts and generates test tokens, and its source is public.

7.2 Frameworks and SDKs

Official MCP SDKs (TypeScript, Python, Go, C# tier-1; Rust beta). The specification has revised five times since late 2024, so the SDK is your upgrade path, not a convenience.

Microsoft Agent Framework (GA 3 April 2026). Target this for new .NET orchestration work: one .NET + Python SDK (Go in preview) with MCP support, A2A interoperability, session state, type safety, filters, and telemetry. Microsoft positions it as the successor to both Semantic Kernel and AutoGen and publishes migration guides (Appendix D), and existing code on those keeps working. A2A 1.0 support was announced as forthcoming at GA, so check its status before you design around it.

First-party APS building blocks

Your MCP server's Layer 2 (Section 3.3) is an APS client. Do not write raw HTTP or hand-roll the SSA JWT assertion.

Resource	What it gives you
<code>aps-sdk-net</code>	The .NET SDK. Authentication, Data Management, Model Derivative, OSS, Construction, Webhooks, and a first-party <code>Autodesk.SecureServiceAccount</code> package
<code>aps-sdk-node</code>	The Node.js/TypeScript SDK, the natural pairing with the TypeScript MCP SDK. Same coverage, including <code>@aps_sdk/secure-service-account</code>
SSA developer guide	The authoritative reference for the SSA identity model, key lifecycle, and assertion claims (Section 7.1)
<code>aps-mcp-auth-examples</code>	Working servers for each Layer 1 / Layer 2 combination (Section 3.3)
<code>mcp-devcon2026</code>	A guided end-to-end build

TABLE 19. FIRST-PARTY AUTODESK RESOURCES FOR THE APS SIDE OF AN MCP SERVER.

Pick the APS SDK matching your MCP SDK; they run in the same process.

Other components you will need

Concern	Options
Vector store	Any mainstream managed or embedded option; the choice matters far less than chunking and hybrid search
Lexical search	BM25 via your search engine of choice - required for hybrid (Section 4.2)
Reranking	A cross-encoder reranker, hosted or self-hosted

Concern	Options
Schema validation	Pydantic (Python), Zod (TypeScript), System.Text.Json + JSON Schema (C#)
Observability	Structured logging and tracing across the agent loop (Section 7.5)

TABLE 20. OTHER INFRASTRUCTURE A RAG/AGENT STACK NEEDS.

Giving an agent an API it cannot call: `dll2llm`

Writing correct code against a large .NET API the model has never seen - the Vault SDK, the Revit API - is a separate problem from calling it.

`dll2llm` reflects over a .NET assembly and emits a modular **Agent Skill** split by namespace, so the agent loads one namespace at a time instead of a multi-million-token reference (Appendix D).

7.3 Model Selection: Hosted, Local, and Model-Agnostic Design

	Hosted frontier model	Local / self-hosted model
Reasoning quality	Best available	Weaker, improving fast
Data	Leaves your walls (subject to the provider's retention and training terms)	Full data sovereignty
Cost model	Per token	Fixed hardware + ops
Per-call cost	Higher	Lower once the hardware exists
Ops overhead	None	Real - you now run inference infrastructure
Context window	Very large (1M tokens on current frontier models)	Typically much smaller

TABLE 21. HOSTED FRONTIER MODELS VERSUS LOCAL MODELS.

The honest answer is usually a mix: small or local models for high-volume, low-risk, bounded tasks; hosted frontier models for the hard reasoning steps.

Be deliberate about what actually leaves your walls. For a Vault MCP server it is not "our CAD files" - it is the item numbers, property values, BOM rows, and file names in tool results, plus whatever the user typed. Write that down for your security review, and keep steps that touch the crown jewels behind a local model.

Architect model-agnostic, because models change monthly and get deprecated.

Put the model call behind an interface, keep provider quirks in the adapter rather than in business logic, keep prompts in versioned files, and keep an evaluation set (Section 4.2) so "is the new model better for us" is a measurement.

Prompts tuned for an older model routinely become counterproductive on a newer one, usually by being over-prescriptive.

7.4 Cost Mechanics

In a naive agent loop that resends the complete, unsummarized history, each turn carries all earlier messages forward. Table 22 illustrates that design. It is not universal: compaction, sliding windows, retrieval, prompt caching, and provider-managed conversation state can change what is sent, processed, or billed.

Turn	What is sent	Result
1	Prompt 1	Answer 1
2	Prompt 1 + Answer 1 + Prompt 2	Answer 2
3	Prompt 1 + Answer 1 + Prompt 2 + Answer 2 + Prompt 3	Answer 3
4	Everything above again, plus Prompt 4	Answer 4

TABLE 22. A FOUR-STEP NAIVE AGENT RUN THAT RE-SENDS THE COMPLETE CONVERSATION ON EVERY TURN.

Without history pruning or compaction, and with roughly similar amounts of new content added per turn, total input tokens across the run grow approximately quadratically with the number of steps. With bounded history, growth can be closer to linear. History growth is therefore a major cost driver in naive loops, not an invariant of every agent design; measure actual provider usage for the implemented architecture.

As of September 2026, current Claude models list approximately \$1 to \$10 per million base input tokens, with output priced at five times input across the listed models. Other providers use different prices and input-to-output ratios; verify the selected provider's current pricing before estimating a deployment.

Six controls that actually work

1. **Prompt caching** is the largest lever, since the system prompt and tool definitions are re-sent every turn. It works on **exact prefix match**, so order stable content first (tools, then system, then messages) and volatile content last. A change to any tool name, description, or schema invalidates the whole cache, which is the most likely way an MCP server breaks its own caching. Verify with the cache-read token count in the usage response; zero across repeated requests means something is invalidating it. **Cacheable MCP list results exist partly to keep this prefix stable** (Section 3.2).
2. **Bound the loop.** Maximum steps, token budget, wall-clock timeout (Section 5.2).
3. **Return less.** Every unnecessary field in a tool result is billed every subsequent turn (Section 3.3).
4. **Aggregate server-side.** The Cockpit's error clustering is roughly a 100-times token reduction (Appendix E.2).
5. **Tier your models.** Cheap, high-volume steps to a smaller model; the frontier model only for the reasoning step that needs it.
6. **Batch what is not interactive.** Overnight drawing checks and corpus re-indexing belong in a batch API at roughly half the price.

Set budgets and caps before you pilot, not after the first invoice (Appendix E.3).

7.5 Governance and the Audit Trail

The question that decides whether this survives a quality audit: *when an AI-assisted MBOM gets approved, who signed off, on what, and can you show it?* Log every one of these, with a correlation ID tying the run together:

What	Why
The user's request, verbatim	The starting point
The resolved prompt - system prompt, tool definitions, retrieved chunks	Reproducing a session is impossible without it
Model and version identifier	Behaviour changes across versions; without this, "it used to work" is unfalsifiable
Every tool call with arguments, and every tool result	The factual basis of the answer
Retrieved chunks with document ID, section, and revision	Proves which version of the standard was applied
The raw model output	Before any post-processing
Validation outcome	What the deterministic guardrails said
The human decision - who, when, what exactly was approved, and what was changed	This is the auditable event. The AI's proposal is an input to a human decision, not a decision
What was ultimately written, by which deterministic system	Closes the loop back to the system of record

TABLE 23. WHAT TO LOG FOR A DEFENSIBLE AUDIT TRAIL, AND WHY.

Three consequences:

- **The system of record stays the system of record.** Vault, Fusion Manage, and the ERP hold the truth; the AI layer holds a decision-support log.
- **Retention.** The AI log needs a retention period consistent with the engineering records it influenced - usually much longer than a default application log policy.
- **Sensitive content lands in logs.** Apply the same access controls to the AI log as to the source systems, or you have built a permissions bypass with excellent search.

Part 8. Getting Started

Where in MY daily work is the repetitive, error-prone, annoying part?

The BOM transformation, the drawing check, the job processor babysitting - that is where you start. Small. Next week.

8.1 Choosing the First Use Case

Score candidates against five criteria. A good first project scores well on all five.

Criterion	Good candidate	Bad candidate
Ambiguity	Genuinely interpretive - no complete rule set exists	Fully specifiable in rules (write the rules instead)
Volume	Happens often enough to matter	Twice a year
Cost of error	Caught by a human before it reaches a system of record	Writes directly to released data
Data quality	The underlying data is already clean and consistent	Depends on properties nobody maintains
Reviewability	A human can verify the output in seconds	Verification takes as long as doing it manually

TABLE 24. FIVE CRITERIA FOR SCORING A FIRST AI USE CASE.

That last row is the one people skip and the one that kills pilots. If checking the AI's work costs as much as doing the work, you have built a demo, not a tool.

8.2 A Sensible Sequence

1. **Clean the data the use case depends on** - not all your data, the specific properties this workflow reads. A missing material property means the drawing checker cannot check the material.

Automation does not fix messy data. It makes messy data faster and louder.

2. **Start with an administrative or read-only assistant** (Section 6.4). An MCP server exposing read-only tools over Vault or your ERP is a useful week-one deliverable; review it against Appendix A before it goes near production data. Lean on industry SDKs, and weigh a local model against a hosted one for anything touching the crown jewels (Section 7.3).
3. **Build the golden evaluation set before the pipeline** (Section 4.2). About 100 real questions from real engineers, with hallucinated citations penalized explicitly.
4. **Add write operations behind a diff**, never as a direct write (Section 5.2). Make the human checkpoint structural, not a sentence in a prompt.
5. **Instrument from the first commit** (Section 7.5). Retrofitting an audit trail is much harder than starting with one.
6. **Measure, then widen**. Accept/dismiss rates, time saved, retrieval recall. Widen only where the numbers support it.

Start narrow, measure, iterate. Define success numerically before you build. Then fix the next one.

Appendix A - MCP Server Review Checklist

Use this when reviewing an MCP server before it goes near production data.

A.1 Tool surface

- Tools are shaped like questions engineers ask, not like REST endpoints or tables
- Analysis and aggregation happen server-side in deterministic code, not in the model
- Every tool description says when to call it, when to call another tool by name, what the defaults are, and what to do with the result
- Human identifiers (part numbers, not GUIDs) are accepted and resolved server-side, and inputs are forgiving
- Every collection result is bounded and paged in the schema, and projected to the fields actually needed
- The server `instructions` field establishes the domain model, a routing policy, normalization rules, and the stakes

A.2 Safety

- Both authentication layers are named explicitly: how the client authenticates to the server, and how the server authenticates to the downstream API (Section 3.3). A local stdio server may operate without Layer 1 authentication; a remote HTTP server accessing protected or user-scoped data authenticates every caller. Session and request-state handles are bound server-side to the verified identity and originating request, expire quickly, rotate when privilege changes, and are protected against replay.
- Read and write are clearly separated, the write surface is minimal, and anything that changes state returns a proposal for confirmation rather than performing the write
- Privileged operations use MCP Apps `visibility: ["app"]` or an MRTR input request, not a prompt instruction - noting the specification does not guarantee that a client answers an input request
- The server authenticates as a real identity with least-privilege permissions, and there is no token passthrough - RFC 8693 token exchange where a downstream credential is needed
- Errors and permission denials return as structured, actionable results, not exceptions
- Write tools are idempotent, and all tool output, retrieved content, and downstream error text are treated as untrusted data
- Outbound URL fetching uses an allowlist, blocks private and link-local destinations, validates every redirect, and does not expose authorization headers to an untrusted destination
- Readable data is minimized to the fields required for the task; network egress and returned model output are controlled so prompt injection cannot silently disclose unrelated readable data

A.3 Operations

- Built on an official MCP SDK against the current specification, over Streamable HTTP for remote and stdio for local

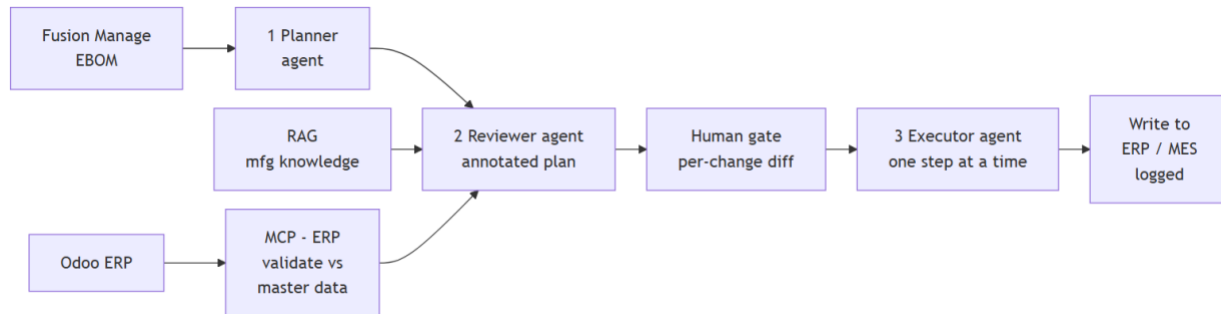
- The tool list is stable between calls, so provider prompt caching is not silently invalidated
- `ttlMs` / `cacheScope` present on every list result and on `resources/read` - the specification requires them, it does not merely allow them
- Every tool call and result is logged with a correlation ID, and timestamps are normalized to UTC
- Logs go to a file, not stdout, if the server can ever run over stdio
- Access fails closed - an empty allowlist denies everyone - and secrets live in a secret store validated at startup

A.4 If the server ships an MCP App

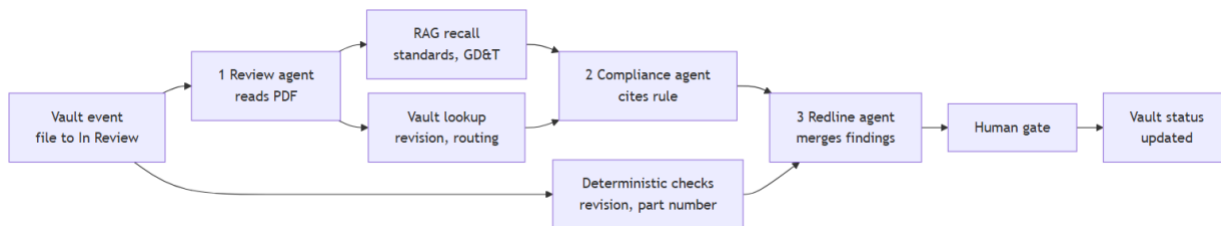
- `connectDomains` and `resourceDomains` list origins only, and are verified against the real host, not a permissive test harness
- No `data:` or `blob: @font-face`; fonts are installed without a `CSS url()` fetch
- The view guards on a non-zero size of both its container and the iframe viewport before constructing anything WebGL
- Any credentialed fetch the iframe needs goes through your own proxy with a short-lived signed ticket
- Embedded third-party telemetry is opted out at the source, not allow-listed in the CSP

Appendix B - Reference Architectures

B.1 MBOM Editor, end to end



B.2 AI Drawing Checker



Compliant drawings move to Released; drawings with findings return to WIP carrying the redlines.

B.3 ForgeFlow change impact analysis



Nothing routes on automatically. The reviewer approves, requests changes, or rejects.

Appendix C - Glossary

Terms not defined where they first appear in the text.

Term	Meaning
Agent Skill	A modular documentation folder an agent loads on demand; the output format of dll2llm
APS	Autodesk Platform Services (formerly Forge)
BM25	A lexical ranking function; strong on exact-token matches
CIMD	Client ID Metadata Documents - the successor to Dynamic Client Registration in MCP auth
EBOM / MBOM	Engineering BOM (by function) / Manufacturing BOM (by process, with routings and operations)
GD&T	Geometric Dimensioning and Tolerancing
Layer 1 / Layer 2	The two authentication layers of an MCP server: client-to-server, and server-to-downstream-API
MCP Apps	The SEP-1865 extension letting servers ship interactive UI via <code>ui://</code> resources
MCP Auth Server	Autodesk's purpose-built OAuth authorization server for MCP flows; not yet publicly documented, currently internal-only
MRTR	Multi Round-Trip Requests - MCP's stateless replacement for server-initiated requests
PKCE	Proof Key for Code Exchange; mandatory for all clients under OAuth 2.1
RRF	Reciprocal Rank Fusion - merges ranked lists by rank rather than score
SSA	Secure Service Account - an APS machine identity
SVF	The APS Viewer's streaming geometry format; generated by Vault for DWF/DWFX
Token exchange	RFC 8693; converting one token into an audience-scoped token for a downstream API
Vault Data API	Vault's built-in REST interface
Vault Gateway	Cloud-mediated outbound access to an on-premise Vault server

TABLE 25. GLOSSARY OF TERMS NOT DEFINED IN THE TEXT.

Appendix D - Links and References

Model Context Protocol

- Specification - <https://modelcontextprotocol.io/specification/>
- 2026-07-28 release notes - <https://blog.modelcontextprotocol.io/posts/2026-07-28/>
- 2026-07-28 release candidate announcement (superseded by the post above) - <https://blog.modelcontextprotocol.io/posts/2026-07-28-release-candidate/>
- Previous stable revision, 2025-11-25 - <https://modelcontextprotocol.io/specification/2025-11-25/>
- Authorization - <https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization>
- Security best practices, token passthrough - https://modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices#token-passthrough
- MRTR pattern (folds in URL-mode elicitation) - <https://modelcontextprotocol.io/specification/2026-07-28/basic/patterns/mrtr>
- SDKs - <https://modelcontextprotocol.io/docs/sdk>
- MCP Inspector - <https://github.com/modelcontextprotocol/inspector>
- SEP-1865, MCP Apps - <https://modelcontextprotocol.io/seps/1865-mcp-apps-interactive-user-interfaces-for-mcp>
- MCP Apps extension spec - <https://github.com/modelcontextprotocol/ext-apps/blob/main/specification/draft/apps.mdx>
- MCP Apps announcement - <https://blog.modelcontextprotocol.io/posts/2025-11-21-mcp-apps/>
- OWASP MCP Security Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/MCP_Security_Cheat_Sheet.html
- RFC 8693, token exchange - <https://datatracker.ietf.org/doc/html/rfc8693>
- RFC 8707, resource indicators - <https://www.rfc-editor.org/rfc/rfc8707.html>
- RFC 9207, OAuth 2.0 authorization server issuer identification - <https://datatracker.ietf.org/doc/html/rfc9207>
- RFC 9728, protected resource metadata - <https://datatracker.ietf.org/doc/html/rfc9728>
- Client ID Metadata Documents (IETF draft) - <https://datatracker.ietf.org/doc/draft-ietf-oauth-client-id-metadata-document/>

Autodesk - MCP and AI

- APS MCP auth examples, all pattern combinations - <https://github.com/autodesk-platform-services/aps-mcp-auth-examples>
- DevCon MCP workshop - <https://github.com/autodesk-platform-services/mcp-devcon2026> · <https://autodesk-platform-services.github.io/mcp-devcon2026/>
- APS MCP App example - <https://github.com/autodesk-platform-services/aps-mcp-app-example>
- Embedding the APS Viewer in AI chats with MCP Apps - <https://aps.autodesk.com/blog/embedding-aps-viewer-ai-chats-mcp-apps>
- dll2llm, .NET assemblies to Agent Skills - <https://github.com/autodesk-platform-services/dll2llm>

- Autodesk Assistant - <https://www.autodesk.com/solutions/autodesk-ai/autodesk-assistant>
- Autodesk Assistant in Vault 2027 - <https://www.autodesk.com/blogs/design-and-manufacturing/meet-autodesk-assistant-in-vault-2027/>
- Prompts to try with Autodesk Assistant in Vault - <https://www.autodesk.com/blogs/design-and-manufacturing/autodesk-vault-ai-assistant/>
- Autodesk Vault 2027.1 delivers a more modern, intelligent Thin Client - <https://www.autodesk.com/blogs/design-and-manufacturing/autodesk-vault-2027-1-delivers-a-more-modern-intelligent-thin-client/> - the source for the 2027.1 Assistant capabilities in Section 7.1
- Autodesk Vault 2027.1 what's new - <https://www.autodesk.com/blogs/design-and-manufacturing/autodesk-vault-2027-1-whats-new/>
- Building for agentic AI: what's new in Autodesk Platform Services - <https://aps.autodesk.com/blog/building-agentic-ai-whats-new-autodesk-platform-services> - the source for Section 6.9
- Autodesk MCP servers - <https://www.autodesk.com/solutions/autodesk-ai/autodesk-mcp-servers>

Autodesk - platform and data

- What's new in Vault 2027 - <https://www.autodesk.com/blogs/design-and-manufacturing/whats-new-in-autodesk-vault-2027-more-insight-more-efficiency-more-impact/>
- Vault Data API overview - https://aps.autodesk.com/en/docs/vaultdataapi/v2/developers_guide/overview/
- Vault Data API reference - <https://aps.autodesk.com/en/docs/vaultdataapi/v2/reference/http>
- Vault Data API samples - <https://github.com/autodesk-platform-services/vault-data-api-samples>
- Vault Data API authentication - https://aps.autodesk.com/en/docs/vaultdataapi/v2/developers_guide/basics/authentication/
- Vault Data API rate limits and quotas - https://aps.autodesk.com/en/docs/vaultdataapi/v2/developers_guide/rate_limits/ - the shared Gateway limit and payload quota (Section 3.3)
- Vault Gateway configuration - <https://help.autodesk.com/view/VAULT/2026/ENU/?guid=GUID-725723AF-CCFD-48A2-8A20-C332B9A75608>
- Vault Gateway FAQ - <https://help.autodesk.com/cloudhelp/2026/ENU/Vault-Admin/files/GUID-0A202D6C-C571-41F7-B62B-42F77A39384D.htm>
- APS Secure Service Accounts - https://aps.autodesk.com/en/docs/ssa/v1/developers_guide/overview/
- SSA JWT assertions - https://aps.autodesk.com/en/docs/ssa/v1/developers_guide/jwt-assertions
- SSA Manager tool - <https://ssa-manager.autodesk.io/> · <https://github.com/autodesk-platform-services/ssa-manager-sample>

- APS authentication - https://aps.autodesk.com/en/docs/oauth/v2/developers_guide/overview/
- APS .NET SDK, with `Autodesk.SecureServiceAccount` - <https://github.com/autodesk-platform-services/aps-sdk-net>
- APS Node.js SDK - <https://github.com/autodesk-platform-services/aps-sdk-node>
- Passing Vault PDFs to the APS Viewer (Tyler Warner) - <https://www.linkedin.com/feed/update/urn:li:activity:7372002940259201026>
- ACC to Forma rebrand - <https://www.autodesk.com/blogs/construction/autodesk-construction-cloud-is-now-autodesk-forma-heres-what-that-means-for-you/>

Frameworks and RAG

- Microsoft Agent Framework - <https://learn.microsoft.com/en-us/agent-framework/overview/>
- Microsoft Semantic Kernel - <https://learn.microsoft.com/en-us/semantic-kernel/overview/>
- MCP tools with Semantic Kernel - <https://devblogs.microsoft.com/agent-framework/integrating-model-context-protocol-tools-with-semantic-kernel-a-step-by-step-guide/>
- Anthropic, Contextual Retrieval - <https://www.anthropic.com/engineering/contextual-retrieval>

Security

- OWASP Top 10 for LLM Applications 2026 - <https://genai.owasp.org/resource/owasp-genai-llm-top-10-2026/>
- OWASP Top 10 for Agentic Applications 2026 - <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>

Products shown

- COOLORANGE - <https://www.coolorange.com>
- powerGate - <https://www.coolorange.com/powergate> · <https://doc.coolorange.com/projects/powergate/>
- powerJobs - <https://www.coolorange.com/powerjobs> · <https://doc.coolorange.com/projects/powerjobsprocessor/>
- IMAGINiT Technologies - <https://www.imaginit.com>
- Symetri ForgeFlow - <https://www.symetri.us/products/forgeflow/>

Appendix E - Developer Reference

Heavy reference material, each subsection pointed to from the section it supports.

E.1 Field Notes from Shipping an APS Viewer MCP App

From Section 3.3. These six field observations come from the authors' implementation and are not normative MCP or Autodesk requirements. Behavior may vary with the APS Viewer version, MCP host, browser, and hosting configuration and should be re-tested after upgrades.

1. Unreadable toolbar.

- Symptom: every toolbar button renders a Latin glyph instead of its icon.
- Cause: the Viewer ships its icons as a `data: @font-face`. `data:` is not an origin, so `resourceDomains` cannot allow it, and the spec's reference CSP hardcodes `data:` into `img-src` and `media-src` but not `font-src`, so a host that applies the default blocks a `data:` web font. A host that applies this CSP without an applicable `font-src` allowance can block the font.
- Fix: install the font without a CSS `url()` fetch. No `resourceDomains` entry rescues it.

2. CSP blocks look like nothing.

- Symptom: the `iframe` fails silently, DevTools shows no error.
- Cause: CSP violations are filed under the console `issue type`, not `error` or `warn`, and blocked requests show as `pending` in the network list, not as failures.
- Fix: check the `issue` panel, not the filtered error console.

3. WebGL draws discarded.

- Symptom: the canvas stays blank; every draw call returns `GL_INVALID_FRAMEBUFFER_OPERATION`.
- Cause: the auto-resize measurement sets the root height to `max-content`, so a percentage or viewport-unit container height resolves to zero, and the canvas is built against a zero-height surface.
- Fix: pin the container to a literal pixel height, and guard on the size of both the container and `document.documentElement` before constructing anything WebGL.

4. Toolbar hidden or app unmounted.

- Symptom: in `fullscreen` display mode the Viewer toolbar sits under the host composer, and dismissing the overlay can unmount the app with no way to reopen it.
- Cause: the Viewer docks its toolbar to the bottom of a `100vh` container.
- Fix: render inline. It also makes problems 2 and 3 visible instead of off-screen.

5. Bugs that only appear in production.

- Symptom: the sizing race and CSP filtering do not reproduce in the test harness.
- Cause: a permissive harness with a fixed panel size applies neither the host's CSP nor its resize behaviour.
- Fix: test against the host you will ship on.

6. Blocked telemetry noise.

- Symptom: `connect-src` blocks calls from a Mixpanel extension you did not load.
- Cause observed in the tested Viewer build: the extension loaded despite `disabledExtensions`.
- Fix: pass `optOutTrackingByDefault: true` to the `Initializer`. Do not add the telemetry host to `connectDomains` - that enables third-party tracking inside the user's session.

E.2 Designing an MCP Tool Surface: A Case Study

From Section 3.4. The **Job Processor Cockpit MCP server** (Section 6.3) is a worked example of a question-shaped tool surface.

Tool	Answers
<code>cockpit-get-health-overview</code>	"Is everything healthy?" - one combined picture
<code>cockpit-find-anomalies</code>	"What got worse?" - period vs. preceding period
<code>cockpit-analyze-failures</code>	"What is failing and why?" - clustered error messages
<code>cockpit-query-jobs</code>	"Show me the failed PDF jobs from yesterday"
<code>cockpit-get-job</code>	Full detail for one job, including parameters
<code>cockpit-get-job-statistics</code>	Counts and rates rather than individual runs
<code>cockpit-list-processors</code>	The machines
<code>cockpit-get-processor-availability</code>	Availability per processor
<code>cockpit-get-processor-performance</code>	CPU, memory, throughput
<code>cockpit-get-busiest-hours</code>	Weekday-by-hour matrix of job starts
<code>cockpit-query-data</code>	Escape hatch for questions the others cannot express

TABLE 26. THE JOB PROCESSOR COCKPIT'S TOOL SURFACE, AND THE QUESTION EACH TOOL ANSWERS.

Four design principles the table does not show:

- **Write descriptions as a navigation graph.** Each says when to call this tool, when to call another, what the default is, and what to do with the result. That is the only place the model reads.
- **Be forgiving about inputs.** `jobType` matches case-insensitively as a substring, so "pdf" finds `coolOrange.Jobs.CreatePDF`. A tool that fails on "pdf" produces a token-burning retry loop.
- **Bound and aggregate before returning.** `limit` is 1-500, default 50, with `offset` for paging. `cockpit-analyze-failures` clusters error messages differing only in ids,

paths, or numbers, so ten thousand strings become twenty clusters - a roughly 100-times token reduction.

- **Use the server instructions field to teach the domain** - domain model, routing policy, normalization rules, and a stakes statement. Copy this stakes line: *"When you recommend an intervention such as restarting a machine, say what evidence supports it and how confident you are. This is production engineering infrastructure and a wrong restart costs the customer work."*

Every Cockpit tool is read-only, timestamps are UTC, and data is scoped to the caller's organization - enforced by the server, not a prompt.

E.3 Risks and Controls

Referenced from Section 3.4 and Section 7.4.

MCP attack vectors

Threat	What it looks like in an engineering context	Control
Confused deputy	A server with ambient authority is talked into an action the requesting user could not perform themselves	Per-action authorization; narrow scopes; act as the <i>user's</i> identity for interactive flows
Token passthrough	Forwarding the caller's upstream token to a downstream API	Forbidden by spec. Use RFC 8693 token exchange to mint an audience-scoped credential
Tool poisoning	A malicious or compromised server hides instructions in a tool description; the model obeys, the user never sees it	Review the full schema of any third-party server before enabling it; put the tool list behind human approval; pin server versions
Indirect prompt injection	A Vault description field, drawing note, email body, or PDF contains "ignore previous instructions and..." and enters context as a tool result	Treat all tool output as untrusted data, never as instructions; keep the human gate on every write; do not let retrieved content alter tool-selection policy
Rogue server registration	An attacker gets a server into the user's client config	Allow-list servers centrally; do not let end users add arbitrary remote servers to a client that also has Vault/ERP access
Excessive agency	The agent reaches tools beyond its task, with broader	Three separate fixes: scope the tool list, scope the

Threat	What it looks like in an engineering context	Control
	permissions than needed, and acts without a human	credentials, keep the approval gate

TABLE 27. SIX REPRESENTATIVE MCP ATTACK VECTORS AND THE CONTROL FOR EACH. THIS IS NOT A COMPLETE THREAT MODEL; SEE SECTION 3.5 AND APPENDIX A.2.

Production risks

Mapped against the OWASP GenAI guidance, the vocabulary your security team will use (Appendix D).

The **LLM** identifiers below are the publicly published 2025 ones; the **ASI** identifiers are the entries of the Agentic Top 10.

Risk from the class	What it looks like here	OWASP anchor	Control
IP / data exposure	What actually leaves your walls when you call a hosted LLM	Sensitive information disclosure (LLM02:2025)	Enumerate the fields appearing in tool results and prompts; scope them; local model for the crown jewels; check retention and training terms
Security & access	SSA vs. 3-legged OAuth; exactly which data the agent can reach	Excessive agency (LLM06:2025); identity and privilege abuse (ASI03)	Least-privilege Vault user; scoped tool surface; no shared admin credentials; token exchange, never passthrough
Cost unpredictability	Agent loops and round trips multiply tokens fast	-	Budgets, caps, step limits, prompt caching (Section 7.4)
Accountability & traceability	Who signed off on an AI-approved MBOM?	-	Log every step; the human approval is the auditable event (Section 7.5)
Vendor lock-in & model churn	Models change monthly; a deprecation breaks your workflow	-	Model-agnostic architecture; versioned prompts; an evaluation set (Section 4.2)
Over-reliance / deskilling	Engineers rubber-stamping AI output instead of reviewing it	Human-agent trust exploitation (ASI09)	Make rejection cheap; review at the level of the individual change; monitor accept/dismiss rates; rotate reviewers;

Risk from the class	What it looks like here	OWASP anchor	Control
			do not measure reviewers on throughput
Prompt injection (direct and indirect)	A Vault description, drawing note, or email contains instructions the model obeys	Prompt injection (LLM01:2025)	Treat all tool output as untrusted data; human gate on writes; architectural least privilege (Table 27)

TABLE 28. PRODUCTION RISKS, OWASP ANCHOR, AND CONTROL.

Over-reliance is the quietest risk and the one your controls will not catch: a rubber-stamped approval looks identical to a considered one in the log. The counter-measures are organizational - keep the review unit small, keep rejection frictionless, and treat a 99% accept rate as a signal to investigate.

E.4 Vault Data API Tool Design

From Section 3.3.

Viewables - two failure modes

In the tested sandboxed MCP App, the Viewer could not call Vault directly.

- Symptom: the sandboxed iframe cannot fetch the derivative.
- Cause: it has no Vault credential, and the Gateway does not send permissive CORS headers.
- Fix: expose a `/svf` route on your own server that forwards to the Vault endpoint, authorized by a short-lived signed ticket minted by the render tool rather than by the user's token. Declare that route in the app's `connectDomains` and `resourceDomains`. Keep the ticket signing key stable across restarts, or a restart invalidates every open viewer and the fix is to re-run the render tool, not reload the iframe.

In the authors' Vault test, an uncached SVF request sometimes triggered on-demand translation.

- Symptom: `Document.load` hangs forever with neither callback firing, on large assemblies only.
- Cause: Vault answers `202 Accepted` while translating; the body is not the manifest. `202` is a `2xx`, so a proxy testing only `IsSuccessStatusCode` streams a placeholder. A part usually has a cached derivative; a large assembly does not.
- Fix: map `202` to a `503` with `Retry-After` and an explicit "translating" body, and have the client retry the manifest. Use a bounded, configurable retry window and measure the required translation time in your own environment; do not assume a fixed duration for every assembly or Vault installation.

Turning that into good tools

Design against the questions engineers actually ask, not one tool per REST endpoint:

Bad (endpoint-shaped)	Good (question-shaped)
<code>get_file_versions(fileId, limit, offset)</code>	<code>find_item(number_or_description)</code> - searches, returns the best matches with the properties people actually want
<code>get_item_bom(itemId, level)</code>	<code>get_bom(item_number, levels=1)</code> - resolves the number, flattens to a bounded depth, returns rows with quantity, UoM, and state
<code>get_property_definitions()</code> + <code>get_properties(entityId)</code>	Fold the definitions into the item result so the model sees "Material": "1.4404", not a GUID
<code>update_lifecycle_state(fileId, stateId)</code>	<code>request_state_change(...)</code> returning a proposal object the UI renders for approval, never a direct write

TABLE 29. ENDPOINT-SHAPED VERSUS QUESTION-SHAPED TOOL DESIGN.

Five Vault-specific rules:

1. **Resolve human identifiers.** Accept "9000-0000", not a `masterId` GUID, and resolve it server-side. Making the model chain a lookup call doubles the round trips.
2. **Bound BOM depth by default.** A `get_bom` defaulting to full explosion returns a 40,000-row structure and blows the context window. Default to one or two levels; expose depth as a bounded parameter.
3. **Project the response.** Return the fifteen fields engineers ask about, not everything the Data API returns; every field is billed on every subsequent turn.
4. **Enforce read/write separation.** Anything changing state or lifecycle returns a proposal for confirmation, not a write. Use MRTR (Section 3.2) or MCP Apps `visibility: ["app"]` (Section 3.3) to make it structural.
5. **Surface permission denials as normal results.** "You do not have access to that folder" is a legitimate answer to relay. A stack trace is not.